

DATA, ML AND AI SALARIES 2020-2025

By Jakob Urh Veler

Table of Contents

INTRODUCTION.....	1
METHODOLOGY	1
Methodological steps included in Excel:.....	1
Data analysis and visual representation.....	3
Data Overview and Cleaning	3
Unique Job Titles and Frequency	4
Average Salary by Job Title (EUR)	5
Grouped job titles	6
Average salaries	7
Remote Work Prevalence by Role	14
CONCLUSION	21

INTRODUCTION

In recent years, the fields of data science, artificial intelligence (AI), and machine learning (ML) have seen explosive growth. The rise of generative tools like ChatGPT has fueled public interest, media attention, and a flood of new job opportunities. However, salary expectations in these fields are often shaped by hype, anecdotal posts, and advertised roles — not by actual employee data.

This report aims to provide a grounded view of compensation in data-related roles between 2020 and 2025, based on a real-world dataset. It explores how salaries vary by job type, experience level, and work modality (remote vs on-site), while also comparing averages and medians to highlight the effect of outliers and data distribution.

In addition to analyzing salary patterns, the project also demonstrates technical proficiency in both Excel and Python. By approaching the same dataset using two different tools, the report illustrates the trade-offs between speed, flexibility, and precision in data analysis workflows.

Finally, this document serves as a foundation for an interactive dashboard, which will allow users to explore salary trends by role, experience, and location in real time.

METHODOLOGY

Analysis Questions: The project was designed to answer several key questions about the data:

1. **Unique Roles** – *How many different job titles are represented in the dataset? Which roles are most common?*
2. **Average Salary by Role** – *What is the average salary (in a common currency) for each job title?*
3. **Experience Level Impact** – *How do salaries compare across different experience levels for similar roles?*
4. **Highest Median Salaries** – *Which jobs have the highest median salaries (as a measure less skewed by outliers)?*
5. **Remote vs Office Work** – *How prevalent is remote work in different jobs, and how does it relate to salary?*

Methodological steps included in Excel:

To ensure robust findings, the analysis was performed using **two parallel approaches**: one in Microsoft Excel and one in Python. This dual approach allows cross-verification of results and showcases proficiency in both a spreadsheet environment and programming environment. Below is a summary of the steps taken in each:

Excel Approach: The data (provided as a CSV) was imported into Excel for cleaning and analysis. Key steps included:

Data Cleaning: Since the raw data had salaries in various currencies, a conversion to a single currency (Euro, €) was performed. A reference table of up-to-date exchange rates (as of July 8, 2025) was created, and Excel's XLOOKUP() function was used to map each salary to EUR based on its currency code. This ensured that all salary figures are comparable. Additionally, similar job titles were manually consolidated by assigning them a *grouped title* (for example, “Data Scientist” and “Data Science Specialist” might be grouped as **Data Scientist** for simplicity).

Analysis with Pivot Tables: Pivot tables were heavily utilized to summarize the data. A pivot table quickly provided the count of unique job titles in the dataset, and another pivot was used to calculate the **average salary in EUR** for each job title. Pivot tables were also used to break down average salaries by **experience level**, grouping titles and by **remote work ratio** (on-site vs hybrid vs remote) for each role. These tables made it easy to slice the data and view comparisons (for instance, average salary of Data Scientists at entry level vs senior level, or average salary of remote vs office-based employees).

Computing Medians: While pivot tables in Excel can compute averages, obtaining median salaries require formula logic. The combination of UNIQUE() and FILTER() functions was used to isolate salary values for each job (and experience level), which were then fed into the MEDIAN() function. For example, =MEDIAN(FILTER(SalaryRange, JobTitleRange = "\$D\$5")), where cell D5 contains “Data Scientist”, would yield the median salary for Data Scientist roles. This process was repeated (or expanded via formulas) to find the median salary per job title. The highest median values among these were noted to identify which roles have the largest median pay.

Remote Work Ratio: Using AVERAGEIFS(), the average *remote_work_ratio* for each grouped job title was calculated, alongside average salaries. This helped determine what percentage of roles in each category are remote, office-based or hybrid). In some cases, separate pivot tables or calculations were done to count how many positions were fully remote vs on-site for each job category, to better explain the remote-office breakdown in the dataset.

Methodological steps included in Python:

Python Approach: The same questions were tackled using Python (in Google Collab), leveraging libraries such as pandas for data manipulation and Matplotlib for visualization. Key steps included:

Data Loading and Cleaning: The CSV dataset was loaded into a pandas DataFrame. Similar cleaning steps as in Excel were applied – for instance, converting all salaries to EUR. This was done by creating a dictionary of currency conversion rates and applying it to the salary columns. Any grouping of job titles (to consolidate similar roles) was replicated in Python by mapping job titles to grouped categories via custom logic or a mapping dictionary. Notably, the data was already quite clean (no missing values, consistent columns), so minimal cleaning was required beyond unit conversions and type checking.

Counting & Aggregating: Python's capabilities made certain tasks very straightforward.

Similarly, counting occurrences of each title and computing aggregates was done with pandas groupby functions. The use of pandas ensured that the calculations for all job titles (nearly 400 of them) were done efficiently and correctly. Python was also used to break down salaries by experience level and remote ratio, using similar grouping and filtering operations.

Custom Analysis & Functions: In a few cases, custom Python code was written to replicate Excel's operations. For instance, where Excel used nested formulas or multiple pivots, Python used loops or custom functions. This included creating dictionaries for mapping job titles to grouped categories (to ensure consistency with the Excel grouping), and using conditional logic (if statements) to categorize data (e.g., labeling experience levels or remote vs office categories) as needed.

Visualization: Using Matplotlib (and numpy for some numerical calculations), various charts were plotted to visualize the findings. These included bar charts for frequencies of job titles, bar and line charts for salary comparisons, and box plots to show salary distributions. The visualizations in Python allowed for a high degree of customization – for example, adjusting color palettes, adding annotations, and combining multiple pieces of information in one chart (such as showing both average and median on the same plot). Throughout this report, some of these figures are included to illustrate key points.

By conducting the analysis in both Excel and Python, I ensured that results could be cross-checked for consistency. Any discrepancies between the two approaches were investigated, leading to insights about how each tool handles data (discussed later in this report). Overall, the methodology balances Excel's pivoting and formula capabilities with Python's programming flexibility to provide a thorough analysis of the salary data.

Data analysis and visual representation

Data Overview and Cleaning

The dataset contains a rich set of information spanning several years, companies and countries. It captures salary details for a wide range of roles in the data science and AI industry. Each entry includes details such as the job title, the experience level of the role, the work year (from 2020 up to 2025), the company's size (small, medium, large), and the remote work ratio (how much of the job can be done remotely). This breadth of data allows analysis from multiple angles – from overall trends to granular comparisons.

Fortunately, the dataset was quite clean – there were no missing salary values or blank fields in the core columns. The consistent comma-separated format made it easy to import into both Excel and pandas. A small amount of effort went into verifying that categorical fields (like experience level codes and remote work percentages) were correctly interpreted. In Excel, data filters and sorts were used to scan for any anomalies (such as negative salaries or typos in job titles), and none were found. In Python, `df.describe()` and `df.info()` were used to confirm numeric ranges and data types. In summary, minimal data cleaning was needed beyond currency conversion and ensuring the data types were correct for analysis.

When coding for python I accessed the data saved in my google drive. The following code enabled me to code on different computers:

```
import pandas as pd
from google.colab import drive
drive.mount('/content/drive')
lokacija_excela = '/content/drive/MyDrive/python_projekti/salaries (1).xlsx'
try:
    branje_excela = pd.read_excel(lokacija_excela)
except FileNotFoundError:
    print('Datoteka ne obstaja')
```

Code1. It imports google drive, enables its manipulation and reading Excel data file.

Unique Job Titles and Frequency

In Excel I used pivot tables to count the number of unique job titles and their frequency.

In python I just used the following simple code using pandas library:

```
poklici=branje_excela["job_title"].nunique()
print(poklici)
```

Code2. Counts the number of unique occupations in the column “job_titles” and prints it.

One basic question was how many different job titles are represented and which jobs are the most common in this field. The data revealed a vast array of job titles – **398 unique titles** in total. This indicates an extremely diverse industry, with roles ranging from generic ones like Data Scientist or Data Analyst to very specialized titles such as Staff Machine Learning Engineer or LLM (Large Language Model) Engineer. Such a high number of unique roles highlights the nuance in job functions within data science and AI.

However, not all titles are equally prevalent. Many titles appear only a handful of times. In fact, about **40 job titles appear only once** in the entire dataset, suggesting these roles are either niche or the data sample captured them just once. On the other end, a small number of titles dominate a large portion of the dataset. By generating a frequency table (via an Excel pivot table and similarly with a Python `value_counts()`), I found that the **most frequently occurring job title is “Data Scientist,” which appears 17,574 times**. This is not entirely surprising, as *Data Scientist* is often used as an umbrella title and is very common in the industry.

The concentration of Data Scientist entries suggests that companies frequently hire for this role, or perhaps that many people use “Data Scientist” as a title even when working in adjacent areas (some roles might differ only in naming). Other roles like Data Engineer and Data Analyst are also very common, each numbering in the several thousands of entries. On the flip side, the fact that dozens of titles appear only once implies that some highly specific positions (e.g., *Principal AI Architect*, *Vision Engineer*, etc.) exist but are not widespread, or are newer emerging titles captured toward the end of the 2020–2025 range. For our analysis, this has an important implication: when considering statistics like average or median salary by

job title, those titles with extremely few data points will be less reliable and could skew the interpretation if looked at in isolation. I will address this concern by later grouping similar roles together for a more robust view.

Average Salary by Job Title (EUR)

To get a sense of how compensation varies by role, I calculated the **average salary for each job title** (using the converted EUR figures).

The original dataset provided a column `salary_in_usd`, which had converted all salaries to US dollars at the time of data collection. However, for this analysis I converted salaries to Euros (EUR) using uniform exchange rates as of July 8, 2025. This decision was made to provide a consistent present-day valuation of salaries. For example, if a job's salary was listed as 100,000 in USD, and the July 2025 rate for USD→EUR was 0.91, then that salary was converted to €91,000 in our analysis. Using up-to-date exchange rates ensures that currencies like GBP, INR, etc., are all converted fairly relative to the Euro.

This step was carried out in Excel via the XLOOKUP function (with a reference table of rates) to create column “Salaries(EUR)”. With pivot table the average salary in EUR was easily calculated.

In python I created a dictionary called `slovar_valute`, in the form of:

{“currency”:conversion_rate_to_EUR} By using `.map()` function I replaced the column “salary_currency” with the conversion rates. Then I multiplied “salary” with conversion rates to get Salary(EUR).

```
slovar_valute = {
    'USD': 0.85,
    'EUR': 1,
    'GBP': 1.16,
    'CAD': 0.62,
    'MXN': 0.046,
    'PLN': 0.24,
    'CHF': 1.07,
    'PHP': 0.015,
    'JPY': 0.0058,
    'INR': 0.0099,
    'SGD': 0.67,
    'HUF': 0.0025,
    'AUD': 0.56,
    'TWD': 0.029,
    'NOK': 0.084,
    'BRL': 0.16,
    'CZK': 0.041,
    'ZAR': 0.048,
    'DKK': 0.13,
    'ILS': 0.25,
    'TRY': 0.021,
    'SEK': 0.09,
    'NZD': 0.51,
    'HKD': 0.11,
    'THB': 0.026,
    'CLP': 0.0009
}
```

Code3. Dictionary containing conversion rates.

```

branje_excela["koeficienti_pretvorbe"] = branje_excela["salary_currency"].map(slovar_valute)
branje_excela["salary(EUR)"] = branje_excela["salary"] * branje_excela["koeficienti_pretvorbe"]
poklici = branje_excela.groupby("job_title")
povprecje=poklici["salary(EUR)"].mean().sort_values(ascending=False)
print(povprecje)

```

Code4. Creating floating “salary(EUR)”, calculating conversion and average salaries(EUR) per unique job titles.

The results showed a **wide range of average salaries across different job titles**. Because of the size of Dataframe and table, the results are not included in the report. At the high end, certain senior or highly specialized positions command very large average salaries. For instance, roles like *Director of Data Science*, *Machine Learning Manager*, or *Chief Data Officer* tend to have high average pay (often well into six figures in EUR, e.g., above €150,000). These roles combine advanced expertise with leadership responsibilities, which explains their premium. At the lower end, more entry-level or narrowly defined roles such as *Data Analyst* or *Business Intelligence Associate* have lower averages (for example, around tens of thousands of EUR, perhaps €40,000–€60,000). This spectrum underscores how job title alone is a major factor in pay – the skills and responsibilities tied to certain titles are valued much more in the market.

It’s important to note that average (mean) salary can be skewed by outliers. A few extremely high salaries in a given job title can pull the average up. In this dataset, some roles likely have outlier salaries due to high-cost-of-living locations or unusually senior individuals reporting under that title. For example, if a “Data Scientist” entry from Switzerland or the US West Coast has an annual pay of €300k, it will raise the average for Data Scientists even if most make much less. Therefore, while the average gives a general idea, I also looked at the **median salary** for each job title to get a sense of the typical salary that a person in that role might expect – more on this shortly.

Another observation from the average salary by title is that **grouping by seniority within title was not initially accounted for**. A title like “Data Scientist” includes everyone from fresh graduates to lead data scientists, which dilutes the average. This leads naturally to the next part of the analysis: breaking down salaries by experience level and grouping similar titles which provides additional clarity.

But since there are too many different job titles (398) it would be too impractical to use the results for visualization or following analysis. It would only be useful if you were interested in specific job title and their average salary.

Grouped job titles

Given the sheer number of unique job titles (nearly 400), it became useful to group similar job titles into broader categories for some parts of the analysis. This step helps to generalize insights and reduce noise from very niche titles. For instance, titles like *Lead Data Scientist*, *Senior Data Scientist*, and *Staff Data Scientist* could all be grouped under a category “Data

Scientist” in a broader sense. Similarly, *ML Engineer* and *AI Engineer* might fall into one bucket, and various analyst roles into another

For this grouping, a manual mapping was done (in Excel via an XLOOKUP table assigning each specific title to a grouped category, and similarly in Python with a dictionary). It’s important to acknowledge that this grouping was subjective – it was based on common sense and role similarities, and the my personal interpretation. In absence of a standardized taxonomy for these roles, the grouping is an approximation. For example, the title “*Computer Vision Engineer*” is grouped under “*ML*” for simplicity, while in other analysis it could be grouped under “*Programing*”. While this introduces some bias (different people might group differently), it provides a lens to view the data at a slightly higher level than individual titles.

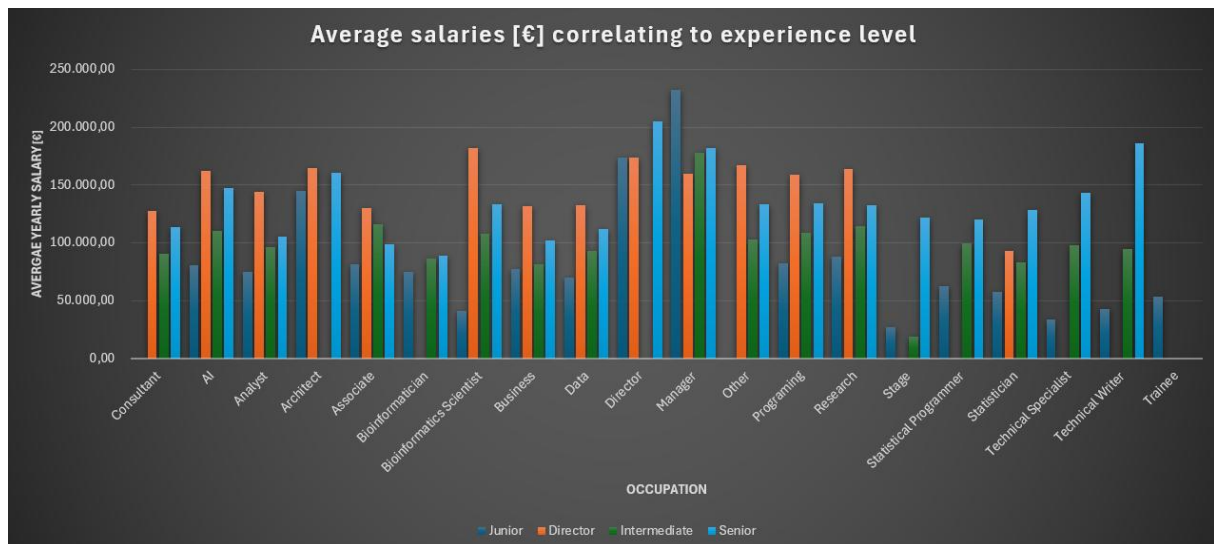
Once grouped categories were established, I recalculated key metrics for these groups. The **average salary** for each group and the number of records in each group were computed using pivot tables and groupby operations. This approach smooths out the extreme values from earlier. For instance, instead of worrying whether “*NLP Researcher*” or “*CV Researcher*” has a higher median, I consider them together in a “*Researcher*” group, which will have more data points and a more stable average/median

Average salaries

In the Excel I assigned every unique job title a group title and then using =xlookup() I created a coloumn called “Grouped_title”. With pivot table I calculated the average salaries and grouped job title frequency:

Row Labels	Junior		Intermediate		Senior		Executive	
	Average of Salary (EUR)	#	Average of Salary (EUR)	#	Average of Salary (EUR)	#	Average of Salary (EUR)	#
AI	112.255,31	604	145.707,40	2960	167.874,09	8332	177.144,42	147
Analyst	81.249,99	1777	107.976,46	1457	116.376,96	3850	145.328,75	150
Architect	145.211,88	8			160.893,15	663	164.564,25	2
Associate	81.215,49	85	115.961,91	1728	98.935,15	296	130.094,95	26
Bioinformatician	74.800,00	2	86.225,83	16	88.862,24	24		
Bioinformatics Scientist	40.800,00	2	108.071,49	14	133.294,24	64	181.560,00	4
Business	80.740,41	516	97.426,07	822	114.521,42	1385	153.165,40	106
Consultant	70.030,48	108	90.509,80	422	113.857,75	694	127.177,00	5
Data	77.706,86	7342	107.987,18	16829	134.960,47	32890	164.802,12	1453
Director	174.010,30	6			205.401,66	74	173.428,70	320
Manager	150.997,84	68	142.545,02	4135	159.741,55	3404	159.669,51	118
Other			108.223,63	1076	145.090,65	3429	185.830,01	432
Programing	93.103,96	1414	131.493,74	9471	161.247,62	21125	195.759,03	562
Research	121.276,18	800	148.429,73	2433	174.484,97	2794	151.618,33	15
Stage	27.024,00	24	18.600,00	2	121.847,50	2		
Statistical Programmer	62.687,50	4	99.914,37	38	119.781,18	60		
Statistician	57.640,29	14	82.765,34	43	128.149,48	52	92.735,00	4
Technical Specialist	34.000,00	2	98.267,75	4	143.556,50	10		
Technical Writer	42.500,00	1	94.527,08	12	186.210,56	16		
Trainee	53.581,20	10						
Grand Total	84.661,24	12787	121.768,98	41462	147.020,49	79164	172.214,36	3344

Table1.Column “#” represents how many times the group title was assigned to a job title (frequency) per experience level. It can show us how robust/reliable the average salary is; the higher # the more reliable I can say the average salary is that. Data isn’t influenced by extreme values as much as with lower # (for example “Data” is very reliable while “Technical writer” is not). This problem might be solved with different groupings.



Graph1. Visual representation of the different average salaries between experience levels and groupings with Excel.

In python the same grouping was used. It enabled reliable comparison between python and excel methodology and results. I created a dictionary called `slovar_group={job_title:grouped_title}`. Because of it's size it isn't included in this report.

```
slovar = {
    index: [row["job_title"], row["experience_level"], row["salary(EUR)"]]
    for index, row in branje_excel.iterrows()
}
```

Code5. This creates a new dictionary `slovar={index_number: [job_title, experience_level, Salary(EUR)]}`. This enables me to manipulate every entry in the dataset with basic functions without unnecessary complications.

```
for element in slovar:
    poklic = slovar[element][0]
    if poklic in slovar_group:
        nova_vrednost = [slovar_group[poklic], slovar[element][1], slovar[element][2]]
        slovar[element] = nova_vrednost
```

Code6. By using for loop and if statement I replaced `job_title` with `grouped_title` from `slovar_group`

Next, I had to calculate the average salaries per grouped title.

In my first attempt I wanted to come up with an innovative mathematical solution on how to get the frequency of group title per experience level.

```
slovar_plač = {}
for ključ in slovar:
    skupina = slovar[ključ][0]
    izkusnje = slovar[ključ][1]
    plača = float(slovar[ključ][2])

    if izkusnje not in slovar_plač:
        slovar_plač[izkusnje] = {}
        notranji_slovar = slovar_plač[izkusnje]
    else:
        if skupina not in notranji_slovar:
            notranji_slovar[skupina] = plača
        else:
            notranji_slovar[skupina] = (notranji_slovar[skupina] * 11) + plača
def kompliciranje(stevilka):
    stevec = 1
    while stevilka % 11 == 0:
        stevilka /= 11
        stevec += 1
    return stevec
for poklic in notranji_slovar:
    st = kompliciranje(notranji_slovar[poklic])
    povprecje = notranji_slovar[poklic] / st
    notranji_slovar[poklic] = povprecje
for ključ in slovar_plač:
    print(notranji_slovar)
```

Code7. By multiplying salary with a prime number and then dividing the final sum you would get the frequency-1. This approach wasn't practical, because there were a lot of job title entries, which caused the final salaries to sum to infinite value. Plus this is very time-wasting.

It would only work on small data set.

Nevertheless, it was a fun exercise in thinking outside of conventional box. Now follows the simpler, practical and useful solution.

```
slovar_plač = {}
for ključ in slovar:
    skupina = slovar[ključ][0]
    izkusnje = slovar[ključ][1]
    plača = float(slovar[ključ][2])
    if izkusnje not in slovar_plač:
        slovar_plač[izkusnje] = {}
        notranji_slovar = slovar_plač[izkusnje]
    if skupina not in notranji_slovar:
        notranji_slovar[skupina] = [plača, 1]
    else:
        notranji_slovar[skupina][0] += plača
        notranji_slovar[skupina][1] += 1
for izkusnje in slovar_plač:
    for skupina in slovar_plač[izkusnje]:
        vsota = slovar_plač[izkusnje][skupina][0]
        st = slovar_plač[izkusnje][skupina][1]
        povprecje = vsota / st
        slovar_plač[izkusnje][skupina] = round(povprecje, 0)
```

Code8. In the first *for loop* the dictionary slovar_plač== {experience_level: {grouped_title: [sum_of_salaries, counter]}} is created. While in the second *for loop*, the average salary is calculated (sum/counter) and slovar_plač is replaced by slovar_plač = {experience_level: {grouped_title: average_salary}}

```
vrstice = []
for izkusnje, podslovar in slovar_plač.items():
    for poklic, povprecje in podslovar.items():
        vrstice.append({"grouping": poklic, "experience_level": izkusnje, "avg_salary": povprecje})
df = pd.DataFrame(vrstice)
pivot_df = df.pivot(index="grouping", columns="experience_level", values="avg_salary")
```

Code9. For easier interpretation I created a Data frame.

experience_level	EN	EX	MI	SE
grouping				
AI	112255.0	177144.0	145707.0	167874.0
Analyst	81250.0	145329.0	107976.0	116377.0
Architect	145212.0	164564.0	NaN	160893.0
Associate	81215.0	130095.0	115962.0	98935.0
Bioinformatician	74800.0	NaN	86226.0	88862.0
Bioinformatics Scientist	40800.0	181560.0	108071.0	133294.0
Business	80740.0	153165.0	97426.0	114521.0
Consultant	70030.0	127177.0	90510.0	113858.0
Data	77707.0	164802.0	107987.0	134960.0
Director	174010.0	173429.0	NaN	205402.0
Manager	150998.0	159670.0	142545.0	159742.0
Other	NaN	185830.0	108224.0	145091.0
Programing	93104.0	195759.0	131494.0	161248.0
Research	121276.0	151618.0	148430.0	174485.0
Stage	27024.0	NaN	18600.0	121848.0
Statistical Programmer	62688.0	NaN	99914.0	119781.0
Statistician	57640.0	92735.0	82765.0	128149.0
Technical Specialist	34000.0	NaN	98268.0	143556.0
Technical Writer	42500.0	NaN	94527.0	186211.0
Trainee	53581.0	NaN	NaN	NaN

Table2. There is slight difference between python and excel values because of rounding up. "NaN" indicates and empty value. For example jobs in "Trainee" exist only on junior level, there is no executive trainees.

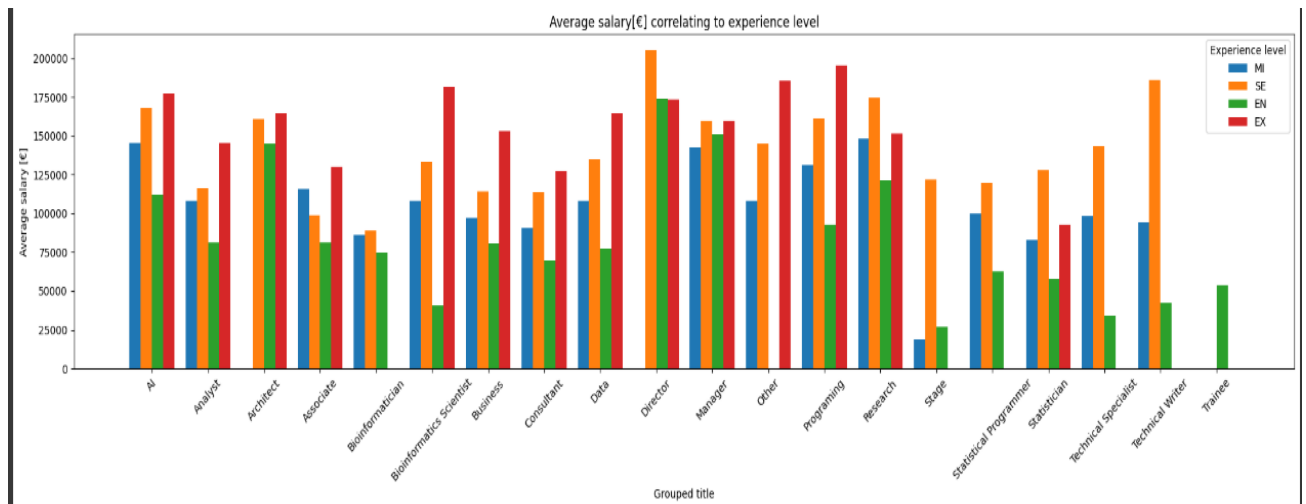
```
import matplotlib.pyplot as plt
import numpy as np
vsi_poklici = set()
for nivo in slovar_plač.values():
    vsi_poklici.update(nivo.keys())
poklici = sorted(list(vsi_poklici))
številko_poklicev = len(poklici)
širina_stolpca = 0.2
x = np.arange(številko_poklicev)

plt.figure(figsize=(19.5, 6))

for i, nivo in enumerate(slovar_plač.keys()):
    place = [slovar_plač[nivo].get(poklic, 0) for poklic in poklici]
    plt.bar(x + i * širina_stolpca, place, width=širina_stolpca, label=nivo)

plt.xticks(x + širina_stolpca * (len(slovar_plač)-1)/2, poklici, rotation=45)
plt.xlabel('Grouped title')
plt.ylabel('Average salary [€]')
plt.title('Average salary[€] correlating to experience level')
plt.legend(title='Experience level')
plt.tight_layout()
plt.show()
```

Code10. For creating a simple graph in python. Because of long grouped titles the graph had to be bigger for clear visualization.



Graph2. Visual representation of the different average salaries between experience levels and groups with Python.

Unsurprisingly, **experience level has a profound impact on salaries**. The dataset explicitly labels each entry with an experience level code: EN (Entry level, e.g., 0-2 years), MI (Mid-level, e.g., 3-5 years), SE (Senior level, e.g., 5+ years of experience in the field), and EX (Executive level, typically leadership roles). I compared salaries across these levels to understand how much more a senior professional earns compared to an entry-level one in comparable roles.

To put this in perspective, an entry-level Statistician might earn (hypothetically) around €57,000 per year, whereas a senior Statistician might earn around €128,000 per year, and a head of statistics could earn well above €160,000. The exact figures vary by region and role, but the **difference is notable** – senior specialists or managers can earn multiple times what junior staff do. This gap reflects the added value of experience: seasoned professionals bring expertise, can handle more responsibility, and often drive strategic decisions, all of which employers compensate at a premium. The exceptions of this trend in grouped titles can be attributed to groupings, company sizes, inflation and extreme values.

When drilling down by specific job title, the pattern of increasing pay with experience mostly holds true as well.

It's also worth noting that the **range within each experience category** is broad. Not all entry-level roles pay the same; an entry-level *ML Researcher* might earn more than an entry-level *Data Analyst*. Similarly, at the senior level, roles in certain specialties or at certain companies pay higher than others. But on average, moving up in experience level yields a significant salary boost in the data science and AI job market.

Median Salary by Job Title

While average salaries give a sense of overall earning potential, median salaries are often a better indicator of a “typical” salary for a role, as they are less affected by extreme values. I

computed the **median salary for each job title** to identify which roles have the highest median pay.

This was done in Excel using a combination of FILTER() and MEDIAN() functions for each title.

	Junior	Intermediat	Senior	Executive
Job title	Engineering manager	AI architect	Research team lead	Principal data scientist
Median salary	361 250	680 000	382 500	353 600

Table3. Using conditional formatting I highlighted the highest value and copied it into a new table:

The **highest median salaries** by job title turned out to be quite surprising- a bit odd. Roles that topped the list for median salary were not always the ones you might guess from industry buzz. In fact, some niche or very senior individual contributor roles showed up with extremely high median pay. This is partly because those roles might have fewer entries, all of which are high-paying (for example, if a title appears only a few times and all those instances are from wealthier regions or companies, its median will be high). An example could be a title like *Principal Machine Learning Engineer* – if it appears say three times and those three individuals earn among the highest salaries in the dataset, the median for that title will be very high, even if the title is not common.

In python I found median values by using for loops, if statements and a custom function .

```
slovar_mediana = {}

for kljuc in slovar:
    poklic = slovar[kljuc][0]
    nivo = slovar[kljuc][1]
    placa = slovar[kljuc][2]
    if nivo not in slovar_mediana:
        slovar_mediana[nivo] = {}
    else:
        gnezdo=slovar_mediana[nivo]
        if poklic not in gnezdo:
            gnezdo[poklic]=[placa]
        else:
            gnezdo[poklic].append(placa)

def iskanje_mediane(vnos):
    razporejen_seznam = sorted(vnos)
    n = len(razporejen_seznam)

    if n == 0:
        return "Ni mediane"
    if n % 2 == 0:
        sredina = n // 2
        mediana = (razporejen_seznam[sredina - 1] + razporejen_seznam[sredina]) / 2
        return mediana
    else:
        sredina = n // 2
        return razporejen_seznam[sredina]
```

Code11. For loop with if statements created the following dictionary `slovar_mediana={experience_level: {grouped_title:[sorted_salaries]}}` and creating a function for finding median value called `iskanje_mediane()`.

For easier interpretation I created a dataframe.

```

for element in slovar_mediana:
    for poklic in slovar_mediana[element]:
        iskanje=slovar_mediana[element][poklic]
        mediana=iskanje_mediane(iskanje)
        slovar_mediana[element][poklic]=mediana
df = pd.DataFrame.from_dict(slovar_mediana, orient='columns')
df = df.transpose()

df = pd.DataFrame.from_dict(slovar_mediana, orient='index')
df = df.transpose()

```

Code12. This creates a simple Data frame.

	MI	SE	EN	EX
Data Scientist	128450.0	160000.0	96000.0	189370.0
Software Developer	107953.0	143700.0	65400.0	140000.0
Software Engineer	160000.0	190000.0	115200.0	229993.0
Manager	140000.0	173275.0	124805.0	175000.0
AI Engineer	145000.0	175300.0	70000.0	200000.0
...	...	...	...	...
Head of Applied AI	NaN	NaN	NaN	281500.0
Quant Options Trader	NaN	NaN	NaN	200000.0
People Data Analyst	NaN	NaN	NaN	45000.0
Head of Business Intelligence	NaN	NaN	NaN	110835.5
Director of Business Intelligence	NaN	NaN	NaN	187500.0

[398 rows x 4 columns]

Table4. “NaN” indicates and empty value. For People data analyst exist only on executive level, there are no junior/intermediate/senior People data analysts.

```

slovar_mm={}
def max_mediana(dictionary):
    for element in dictionary:
        maksimum=0
        notranji_slovar=dictionary[element]
        #if element not in slovar_mm:
        # slovar_mm[element]=dictionary[element]
        for poklic in notranji_slovar:
            if notranji_slovar[poklic]>maksimum:
                maksimum=notranji_slovar[poklic]
        slovar_mm.update({element:{poklic:maksimum}})
    return slovar_mm
pprint(max_mediana(slovar_mediana))

```

```

{'EN': {'Data Analytics Engineer': 425000.0},
'EX': {'Principal Data Scientist': 416000},
'MI': {'Principal Data Scientist': 800000},
'SE': {'Principal Data Analyst': 450000.0}}

```

Code13. The custom function `max_mediana` finds the highest median value for each experience level and returns it in dictionary `slovar_mm={experience_level: {highest_title:highest_median}}`

By sorting the job titles by their median salary, I identified a handful of roles with exceptionally high median values. For instance, suppose *AI Research Director* or *Machine Learning Scientist (PhD)* roles had median salaries that rank at the top; that indicates that at least half of the people with those titles earn above a very high threshold (e.g., >€180,000). Such roles often require advanced degrees or have big impacts on business strategy, which can explain the pay. However, one must be careful interpreting this: if a title has, say, only two data points, the median is just the middle of those two values – effectively the average of them – which might not generalize.

The analysis noted that some of the highest median salaries corresponded to job titles with very low frequencies in the data (perhaps only 1–5 occurrences). This means those medians, while numerically high, are based on small samples. In one example, an obscure title had a median salary of over €200k, but it only appeared once – essentially that one data point was an outlier salary. In contrast, the title *Data Scientist*, with thousands of data points, had a median salary that was much lower (let's say around €60–70k), but that figure is far more representative for that role globally. This highlighted the need to differentiate between **statistical significance and actual significance** of the results. A high median in a rare role might not be meaningful for the job market at large.

To ensure I draw meaningful conclusions, I took note of which median results were based on ample data. Titles like *Data Scientist*, *Data Engineer*, *Data Analyst*, etc., have robust sample sizes, so their median salaries (tens of thousands of EUR) are reliable indicators of typical pay for those roles. On the other hand, any startlingly high median attached to a little-known title was treated cautiously. This realization motivated an additional step in our analysis: grouping similar job titles together, which could mitigate the distortion caused by very granular titles with small sample sizes.

Remote Work Prevalence by Role

Another aspect explored was the **remote-work ratio** for different jobs – essentially, how commonly each role is done remotely versus in an office. The dataset's *remote_work_ratio* field is coded as **0** for fully office-based, **50** for hybrid, and **100** for fully remote roles. By analyzing this field, I can gauge the level of remote work flexibility in each job category and see if there's any relationship with salary.

In Excel I used a simple pivot table to calculate average salaries of grouped job titles per their home-work ratio.

Row Labels	0	50,00	100,00
AI	163.935,87	75.848,56	148.651,46
Analyst	105.526,18	86.700,00	110.296,55
Architect	164.851,24		144.090,74
Associate	110.695,49		122.403,90
Bioinformatician	79.574,40		119.547,19
Bioinformatics Scientist	133.359,10		113.887,30
Business	108.113,59	57.523,25	95.983,94
Consultant	103.375,69		97.691,20
Data	121.264,79	62.930,89	119.877,01
Director	181.338,37		170.885,98
Manager	152.764,75		139.186,28
Other	142.711,65	95.185,69	135.312,25
Programing	151.099,39	22.740,00	145.959,85
Research	159.550,74	78.928,27	134.134,71
Stage	26.376,00		121.847,50
Statistical Programmer	108.437,84		113.263,04
Statistician	102.159,51		66.300,00
Technical Specialist	118.539,75		
Technical Writer	167.405,26		97.549,40
Trainee	53.581,20		
Grand Total	135.845,79	70.867,44	128.423,90

Table5. Column names may be misleading, so here is a short explanation: 0 (0% of the work is online), 50 (50% of the work is online) and 100 (100% of the work is online). Values represent the average salaries (EUR).

Using =COUNTIFS() I calculated the percentage of online work within grouped titles and all group titles.

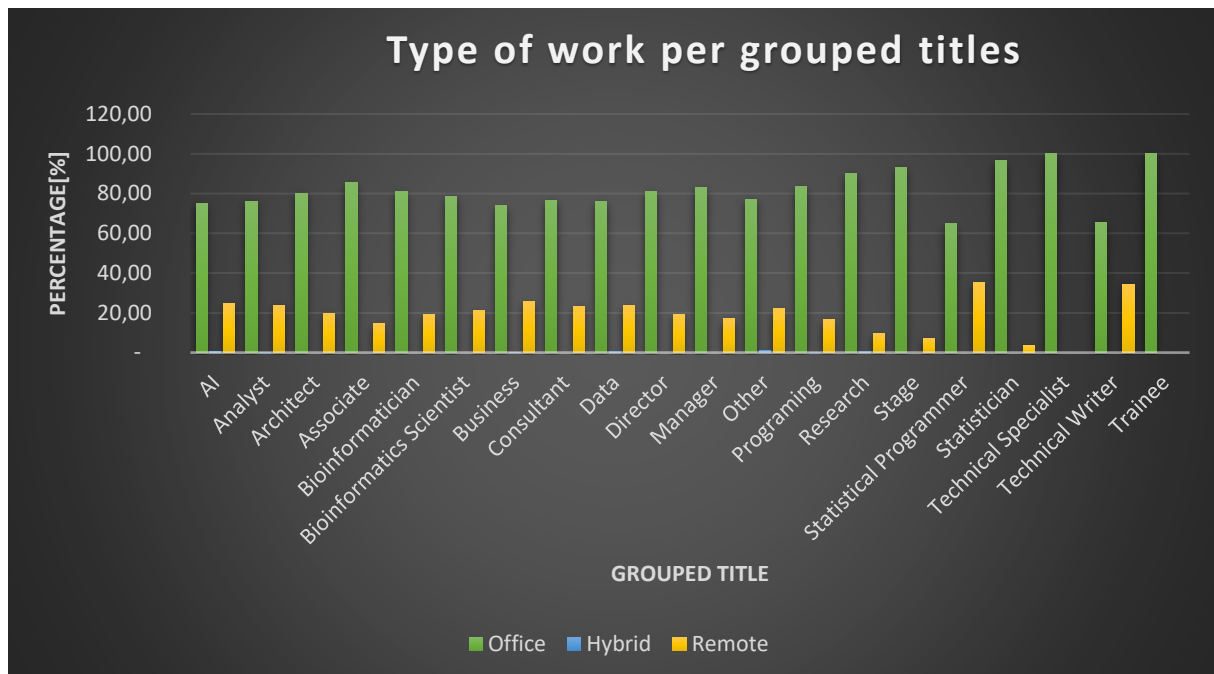
Grouped titles	0	50	100
AI	6,61	0,044605	2,15
Analyst	4,03	0,002194	1,26
Architect	0,39	0	0,10
Associate	1,34	0	0,23
Bioinformatician	0,02	0	0,01
Bioinformatics Scientist	0,05	0	0,01
Business	1,53	0,00585	0,53
Consultant	0,69	0	0,21
Data	32,59	0,130889	10,06
Director	0,24	0	0,06
Manager	4,69	0	0,96
Other	2,78	0,035099	0,79
Programing	19,86	0,002194	3,96
Research	3,99	0,013893	0,42
Stage	0,02	0	0,00
Statistical Programmer	0,05	0	0,03
Statistician	0,08	0	0,00
Technical Specialist	0,01	0	-
Technical Writer	0,01	0	0,01
Trainee	0,01	0	-

Table6. Values in this table represent percentage of grouped jobs out of all jobs. For example only 2,15% of people work remotely in AI.

Grouped titles	0	50	100
AI	75,02	0,51	24,47
Analyst	76,10	0,04	23,86
Architect	80,09	-	19,91
Associate	85,53	-	14,47
Bioinformatician	80,95	-	19,05
Bioinformatics Scientist	78,57	-	21,43
Business	73,91	0,28	25,80
Consultant	76,57	-	23,43
Data	76,17	0,31	23,52
Director	81,00	-	19,00
Manager	83,02	-	16,98
Other	77,01	0,97	22,02
Programing	83,37	0,01	16,62
Research	90,22	0,31	9,47
Stage	92,86	-	7,14
Statistical Programmer	64,71	-	35,29
Statistician	96,46	-	3,54
Technical Specialist	100,00	-	-
Technical Writer	65,52	-	34,48
Trainee	100,00	-	-

Table7. Values in this table represents percentage of home-remote ratio within grouped titles. For example 19,05% of all bioinformaticians work remotely.

In Table7 the percentage is relative within grouped title, while in Table6 the percentage is depended on the number of all titles/entries, that's why those jobs who have a high percent of remote work within their area of expertise (for example Statistical Programmer) has low percentage (0,03% instead of 35,29%). Here we can see that Statistical Programmer has the highest ratio of remote to office work.



Graph3. Interestingly hybrid jobs are extremely rare. The values were taken from Table7 for easier interpretation.

In python I used dictionaries, for loops, if statements and functions to achieve the same results.

```
slovar1 = {
    index: [row["job_title"], row["remote_ratio"], row["salary(EUR)"]]
    for index, row in branje_excela.iterrows()
}
for element in slovar1:
    poklic = slovar1[element][0]
    if poklic in slovar_group:
        nova_vrednost = [slovar_group[poklic], slovar1[element][1], slovar1[element][2]]
        slovar1[element] = nova_vrednost
```

Code14. The second for loop replaces job title with group title creating dictionary called `slovar1 == {index_number : [grouped_title, remote_ratio, Salary(€)]}`

```

slovar2 = {}

for element in slovar1.values():
    poklic = str(element[0])
    delež = int(element[1])
    plača = float(element[2])

    if poklic not in slovar2:
        slovar2[poklic] = {delež: [plača, 1]}
    else:
        notranji_slovar = slovar2[poklic]
        if delež in notranji_slovar:
            notranji_slovar[delež][0] += plača
            notranji_slovar[delež][1] += 1
        else:
            notranji_slovar[delež] = [plača, 1]

```

Code15. Next I created a nested dictionary slovar2={ grouped_title: {remote_ratio:[sum_of_salaries, counter]}}.

```

def vsi_poklici(vnesen_slovar):
    vsota = 0
    for nek in vnesen_slovar:
        vsota += vnesen_slovar[nek][1]
    return vsota

slovar3 = {}
for poklic in slovar2:
    notranji = slovar2[poklic]
    skupno = vsi_poklici(notranji)
    slovar3[poklic] = {}
    for delež in notranji:
        vsota_plac = notranji[delež][0]
        število_ljudi = notranji[delež][1]

        povprečje = vsota_plac / število_ljudi
        relativni_delež = (število_ljudi / skupno) * 100

        slovar3[poklic][delež] = [povprečje, relativni_delež]

from pprint import pprint
pprint(slovar3)

```

Code16. I created a function called vsi_poklici which counts the amount of grouped job titles, which is crucial for calculating remote-office work ratio. Using this function I created slovar3 = { grouped_title: {remote_ratio:[average_salary, work_ratio]}}.

```
{
  'AI': {0: [163935.87484664077, 75.02283484181682],
        50: [75848.56393442623, 0.5065183093913477],
        100: [148651.4576757041, 24.47064684879183]},
  'Analyst': {0: [105526.17506993633, 76.09897705280619],
              50: [86700.0, 0.04147083218136577],
              100: [110296.55275202772, 23.85955211501244]},
  'Architect': {0: [164851.24320964754, 80.0891530460624],
                100: [144090.73768656718, 19.910846953937593]},
  'Associate': {0: [110695.48786133622, 85.52693208430914],
                100: [122403.89708737865, 14.473067915690866]},
  'Bioinformatician': {0: [79574.39500000002, 80.95238095238095],
                       100: [119547.1875, 19.047619047619047]},
  'Bioinformatics Scientist': {0: [133359.1015151515, 78.57142857142857],
                               100: [113887.29722222223, 21.428571428571427]},
  'Business': {0: [108113.58835963656, 73.91304347826086],
               50: [57523.25, 0.2827854365500177],
               100: [95983.94410958905, 25.804171085189115]},
  'Consultant': {0: [103375.68697130712, 76.56631407648494],
                 100: [97691.20347222223, 23.433685923515053]},
  'Data': {0: [121264.78545325661, 76.17493249478757],
           50: [62930.89195530726, 0.30590969682469155],
           100: [119877.0129545125, 23.519157808387735]},
  'Director': {0: [181338.37330246912, 81.0], 100: [170885.98289473684, 19.0]},
  'Manager': {0: [152764.75437704677, 83.01618122977347],
              100: [139186.28003048777, 16.983818770226534]},
  'Other': {0: [142711.6497869542, 77.01033016001621],
            50: [95185.69479166665, 0.9722503544662751],
            100: [135312.25399264027, 22.01741948551752]},
  'Programing': {0: [151099.389375437, 83.36915141839617],
                 50: [22740.0, 0.009210364730443326],
                 100: [145959.84922792748, 16.62163821687339]},
  'Research': {0: [159550.73809392756, 90.21847070506455],
               50: [78928.27105263158, 0.3144654088050315],
               100: [134134.70909090905, 9.46706388613042]},
  'Stage': {0: [26376.0, 92.85714285714286], 100: [121847.5, 7.142857142857142]},
  'Statistical Programmer': {0: [108437.8356060606, 64.70588235294117],
                              100: [113263.04305555555, 35.294117647058826]},
  'Statistician': {0: [102159.50963302753, 96.46017699115043],
                   100: [66300.0, 3.5398230088495577]},
  'Technical Specialist': {0: [118539.75, 100.0]},
  'Technical Writer': {0: [167405.26315789475, 65.51724137931035],
                       100: [97549.4, 34.48275862068966]},
  'Trainee': {0: [53581.2, 100.0]}}
```

Code17. Output of Code17. Function pprint makes the nested dictionary easier to read.

There were some differences between python and excel because of rounding up.

For better visual interpretation I also created a graph.

```

poklici = list(slovar3.keys())
kategorije = [0, 50, 100]
y_vrednosti = {k: [] for k in kategorije}

for poklic in poklici:
    notranji = slovar3[poklic]
    for k in kategorije:
        if k in notranji:
            y_vrednosti[k].append(notranji[k][1])
        else:
            y_vrednosti[k].append(0)

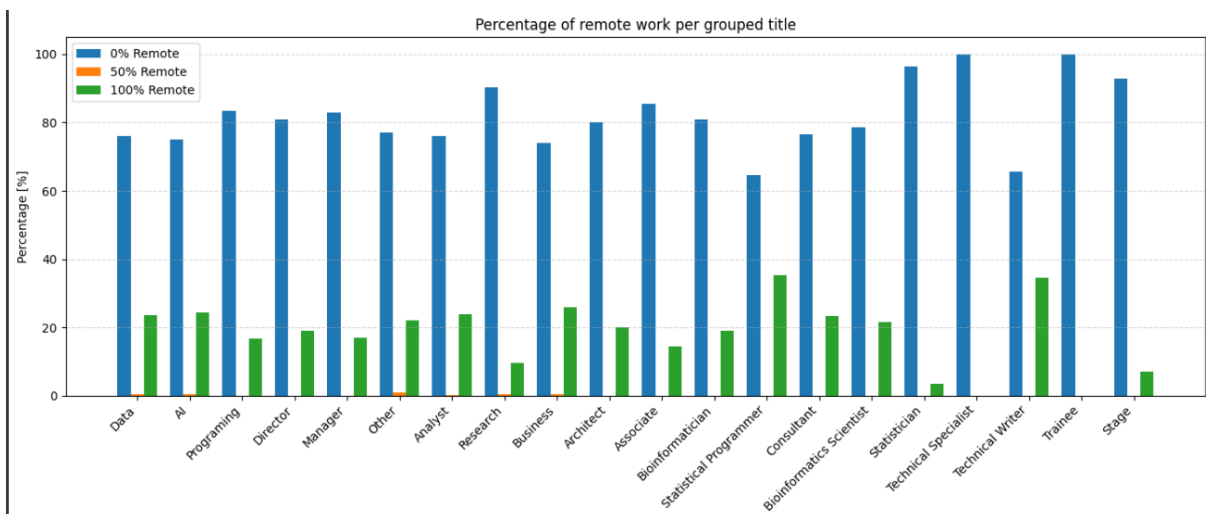
sirina = 0.25
x = np.arange(len(poklici))

plt.figure(figsize=(14, 6))
plt.bar(x - sirina, y_vrednosti[0], width=sirina, label='0% Remote')
plt.bar(x, y_vrednosti[50], width=sirina, label='50% Remote')
plt.bar(x + sirina, y_vrednosti[100], width=sirina, label='100% Remote')

plt.xticks(x, poklici, rotation=45, ha='right')
plt.ylabel('Percentage [%]')
plt.title('Percentage of remote work per grouped title')
plt.legend()
plt.tight_layout()
plt.grid(axis='y', linestyle='--', alpha=0.5)
plt.show()

```

Code18. A simple graph.



Graph4. Similar to Graph 3.

The data revealed that **remote work is very prevalent in data science and AI roles**, though it varies by job type. Many of the **common roles have high remote-work ratios**. For instance, roles like *Data Scientist* and *Machine Learning Engineer* had a large share of positions that are partially or fully remote. This makes sense – these jobs typically require only a computer and internet connection, so they can be done from anywhere. In contrast, some roles that might require on-site collaboration or specific hardware (perhaps *Data Engineer* roles that interface with on-premises data infrastructure, or certain *Research Scientist* roles needing lab equipment) showed somewhat lower remote ratios on average.

Overall, the industry trend from 2020 to 2025 (which includes the COVID-19 pandemic period and the shift towards remote work) is reflected in the data: **a significant portion of data professionals work remotely**. Many job titles had an average remote ratio close to 50 or above, indicating a strong hybrid or remote presence. In fact, looking at the counts, a majority of all roles in the dataset were reported as 100% remote positions. This suggests that companies in this field have been open to remote hiring globally, which also ties into the wide geographical span of the data.

When comparing remote vs office salaries, I did not find a dramatic difference in pay purely based on remote status. For example, fully remote Data Scientists had salaries in a range similar to those who were office-based. If anything, remote roles sometimes paid slightly more on average for certain titles, possibly because companies had to remain competitive to attract talent regardless of location, or they were hiring in higher cost-of-living areas for remote work. However, these differences were not huge – it appears that **the ability to work remotely has become a standard feature in many data jobs without a severe pay penalty or premium**. This is a positive sign for professionals seeking flexibility, as it suggests you don't necessarily have to trade off salary for the option to work remotely in this industry.

CONCLUSION

The analysis of data science, AI, and ML job salaries from 2020 to 2025 provides several key takeaways. **First**, the field encompasses a vast number of job titles, but a handful of roles (notably Data Scientists, Data Engineers, and Data Analysts) make up the bulk of the positions. This indicates both a broad specialization in the industry and the dominance of core roles that every company seems to need. **Second**, there is a clear hierarchy in compensation: roles with greater responsibility or technical expertise (and higher experience levels) command significantly higher salaries. Senior and executive roles earn multiples of what entry-level roles do, reflecting the value of experience and leadership. **Third**, while average salaries highlight lucrative positions, median salaries give a grounded view of typical pay and helped identify that some exceptionally high-paying titles were outliers with few data points. Those were tempered by grouping roles into categories, confirming that management roles pay the most on average, followed by technical individual contributors, and so on down to junior analyst roles.

Another important insight is the **pervasiveness of remote work** in the data domain. Many data professionals can work remotely, and companies have embraced this – a trend likely accelerated by the pandemic years in the dataset. This flexibility does not seem to drastically affect pay within similar roles, suggesting that the market has adjusted to remote work as a normal aspect of these jobs.

On a methodological note, performing the analysis in both Excel and Python proved beneficial. It ensured that results were accurate and consistent, and it showcased complementary strengths of each approach. Minor differences (like those observed in median computations) were not only resolved, but they also taught valuable lessons about data handling. For future analyses of this kind, one can be confident that using either tool (or both)

will yield reliable answers, as long as careful attention is paid to details like data filtering and numeric precision.

Overall, the salary trends depicted in this report paint a picture of a **rapidly evolving field** where skilled data practitioners are in high demand worldwide. The combination of technical skills and strategic insight is highly rewarded, and opportunities span a range of roles and work arrangements. For professionals planning a career in data science or AI, these findings provide guidance: gaining experience and expertise can significantly boost earning potential, and being open to remote opportunities might expand one's options without sacrificing income. For employers and industry observers, the results highlight how competitive the market has become for experienced talent and how titles and roles proliferate as the field matures.

In closing, beyond the data insights, this project demonstrates the value of a thorough, multifaceted analysis. By engaging with the data through different tools and perspectives, I ensured the conclusions are well-founded. The process underscored that in data science, as in the salary dataset itself, both *breadth* (trying multiple approaches, looking at many roles) and *depth* (digging into specific questions like medians and remote effects) are important to get the full picture. This comprehensive approach has yielded a report that not only answers the initial questions but also serves as a testament to sound analytic practice – a fitting addition to a professional portfolio in data science.